

Concurrencia (1er. parcial, evaluación continua) (Solución)

Curso 2024–2025 - 2º semestre (junio 2026)

Grado en Ingeniería Informática / Grado en Matemáticas e Informática

UNIVERSIDAD POLITÉCNICA DE MADRID

NORMAS: Este es un cuestionario que consta de 9 preguntas. Todas las preguntas son de respuesta simple excepto la pregunta 9 que es una pregunta de desarrollo. La puntuación total del examen es de 10 puntos. La duración total es de **60 minutos**. **Sólo hay una respuesta válida a cada pregunta de respuesta simple**. Toda pregunta en que se marque más de una respuesta se considerará incorrectamente contestada. Las respuestas deben consignarse en el casillero de la derecha. Toda **pregunta incorrectamente contestada restará** del examen una cantidad de puntos igual a la puntuación de la pregunta dividida por el número de alternativas ofrecidas en la misma. No olvidéis **rellenar vuestros datos**.

RESPUESTAS

1	2	3	4
5	6	7	8

Cuestionario

- (1 punto) 1. Dado el siguiente programa concurrente, donde se ha eliminado el manejo de excepciones para hacerlo más legible:

<pre>class P1 extends Thread { public void run() { Thread p = new P2(); p.start(); // OJO!! System.out.print("X"); } } class P2 extends Thread { public void run() { System.out.print("Y"); } }</pre>	<pre>class Threads { public static void main(String[] as) { Thread p1 = new P1(); Thread p2 = new P2(); p1.start(); p2.start(); p2.join(); p1.join(); System.out.print("Z"); } }</pre>
---	--

Se pide marcar la afirmación correcta:

- (a) ☐ Y nunca es el primer carácter que se imprime y Z siempre es el último.
 (b) ☒ La salida puede comenzar por X o por Y.
 (c) ☐ Y siempre es el primer carácter que se imprime y Z siempre es el último.

- (1 punto) 2. Dado el programa de la pregunta 1:

Se pide marcar cuál es el número máximo de hilos que podrían ejecutar simultáneamente:

- (a) ☐ 3
 (b) ☐ 2
 (c) ☒ 4

- (1 punto) 3. A continuación se muestra un fragmento de código, con varias declaraciones y unas sentencias numeradas para poder hacer referencia a ellas:

<pre>volatile int a; volatile boolean cond; Semaphore s; int b;</pre>	<pre>1: a++; 2: cond = !cond; 3: s.await(); 4: if (cond) --b;</pre>
---	---

Señala la única afirmación correcta:

- (a) ☐ Las sentencias 1, 2, 3, 4 son atómicas.
 (b) ☐ Las sentencias 1, 2 son sentencias atómicas.
 (c) ☒ La sentencia 3 es la única sentencia atómica.
 (d) ☐ Ninguna de las cuatro son sentencias atómicas.

- (1 punto) 4. Dado un programa concurrente en el que tres hilos instancias de las clases A, B y C comparten una variable m:

```
static volatile int m = 0;
static Semaphore s1 = new Semaphore(1);
static Semaphore s2 = new Semaphore(0);
```

<pre>class A extends Thread { public void run() { s1.await(); m = 3 * m; s1.signal(); } }</pre>	<pre>class B extends Thread { public void run() { s2.await(); s1.await(); m = m * m; s1.signal(); } }</pre>	<pre>class C extends Thread { public void run() { s1.await(); m += 3; s2.signal(); s1.signal(); } }</pre>
---	---	---

¿Cuál es el comportamiento del programa tras terminar los tres procesos?

- (a) ☐ Puede producirse un interbloqueo o acabar en una potencia de 3.
 (b) ☒ Siempre acaba sin error y m valdrá 9, 27 o 81.
 (c) ☐ Siempre se produce un interbloqueo.
 (d) ☐ No se puede saber porque podrían producirse violaciones de la exclusión mutua.

- (1 punto) 5. Se presenta esta implementación de una solución al problema de la exclusión mutua mediante *espera activa*:

```
static volatile boolean quiere_inc = false;
static volatile boolean quiere_dec = false;
static volatile int contador = 0;
```

<pre>class Inc extends Thread { public void run() { for (int j = 0; j < N; j++) { quiere_inc = true; while (quiere_dec) { quiere_inc = false; quiere_inc = true; quiere_inc = false; quiere_inc = true; } contador++; quiere_inc = false; } } }</pre>	<pre>class Dec extends Thread { public void run() { for (int l = 0; l < N; l++) { quiere_dec = true; while (quiere_inc) { quiere_dec = true; quiere_dec = false; quiere_dec = true; quiere_dec = false; } contador--; quiere_dec = false; } } }</pre>
--	--

Suponiendo que tenemos un hilo de la clase Inc y otro de la clase Dec, **se pide** marcar la afirmación correcta:

- (a) ☐ El programa no garantiza la propiedad de ausencia de interbloqueo.
 (b) ☒ El programa no garantiza la exclusión mutua en el acceso a las secciones críticas (contador++ y contador--).
 (c) ☐ El programa presenta un problema de alternancia estricta.

(1 punto) 6. Dado el siguiente código:

```
static Semaphore s1 = new Semaphore(1);
static Semaphore s2 = new Semaphore(1);
```

```
class P1 extends Thread {
    public void run() {
        s1.await();
        s2.await();
        sec_critica_A();
        s1.signal();
        s2.signal();
    }
}
```

```
class P2 extends Thread {
    public void run() {
        s2.await();
        s1.await();
        sec_critica_B();
        s2.signal();
        s1.signal();
    }
}
```

Suponiendo que tenemos un proceso de clase P1 y otro de clase P2, **se pide** marcar la afirmación correcta.

- (a) ☐ El programa no garantiza la exclusión mutua en el acceso a las secciones críticas.
- (b) ☐ El programa garantiza la exclusión mutua, pero sufre de alternancia estricta.
- (c) ☐ El programa garantiza la exclusión mutua en el acceso a las secciones críticas (sec_critica_A y sec_critica_B) y la ausencia de interbloqueos.
- (d) ☒ El programa garantiza la exclusión mutua en el acceso a las secciones críticas (sec_critica_A y sec_critica_B) pero puede sufrir de interbloqueo.

(1 punto) 7. Se tiene un CTAD que almacena diferentes temperaturas para cada mes (por ejemplo, en determinados días y a diferentes horas). A continuación mostramos parte de la especificación, que incluye la POST de una acción llamada o. (Hemos usado la notación s_i para referirnos al i -ésimo elemento de la secuencia s.)

C-TAD Temperaturas

....
ACCIÓN o : **Secuencia**($\mathbb{R}$)[s]

....
TIPO Temperaturas = Mes $\rightarrow$ **Secuencia**($\mathbb{R}$)
DONDE Mes = {1 ... 12}

....
o(s)

POST: Longitud(s) = 12 $\wedge$

$\forall m \in \text{Mes} \bullet$

$(\exists i \in \{1 \dots \text{Longitud}(\text{self}(m))\} \bullet \text{self}(m)_i = s_m \wedge$

$\forall j \in \{1 \dots \text{Longitud}(\text{self}(m))\} \bullet \text{self}(m)_j \geq s_m)$

¿Qué se ha indicado que calcula la acción o?

- (a) ☐ Devuelve los máximos de las temperaturas de cada mes.
- (b) ☒ Devuelve los mínimos de las temperaturas de cada mes.
- (c) ☐ Devuelve el máximo de todas las temperaturas almacenadas.
- (d) ☐ Devuelve el mínimo de todas las temperaturas almacenadas.

(1 punto) 8. Se tiene el siguiente CTAD:

C-TAD Circuit

ACCIÓN op1:

ACCIÓN op2:

ACCIÓN op3:

DOMINIO

TIPO Circuit = Lado $\rightarrow \mathbb{B}$

DONDE Lado = l | r

INICIAL: $\forall j \in \text{Lado} . \text{self}(j) = \text{Falso}$

INVARIANTE: $\neg \text{self}(l) \vee \neg \text{self}(r)$

CPRE: $\neg \text{self}(l)$

op1()

POST: $\text{self} = \text{self}^{\text{pre}} \oplus \{r \rightarrow \neg \text{self}^{\text{pre}}(r)\}$

CPRE: $\text{self}(l) \vee \text{self}(r)$

op2()

POST: $\text{self} = \{l \rightarrow \neg \text{self}^{\text{pre}}(l), r \rightarrow \neg \text{self}^{\text{pre}}(r)\}$

CPRE: Cierto

op3()

POST: $\text{self} = \text{self}^{\text{pre}} \oplus \{l \rightarrow \text{Cierto}\}$

Tres procesos invocan repetidamente las operaciones op1(), op2() y op3()

¿Cómo se comporta esta especificación?

- (a) ☒ Podría llegar a violarse el invariante.
- (b) ☐ Siempre se cumplirá la invariante.
- (c) ☐ El sistema puede quedar bloqueado.

- (2 puntos) 9. En el aeropuerto de Bribonia se ha puesto en marcha un servicio de powerbanks para cargar dispositivos electrónicos para que aguanten vuelos transoceánicos y así tener más pasajeros que su país vecino Truhania. Por limitaciones de capacidad, únicamente se permite la carga cuando no se alcance el límite de carga de $maxpower = 2000kW$.

Para simplificar, los dispositivos se clasifican como de *bajo consumo* (por ejemplo, móviles o tablets) que cargan a 5kW, y los de *alto consumo* (por ejemplo, portátiles), que cargan a 12kW.

No se puede acceder a *cargar* un dispositivo si, al enchufarlo, la potencia de este, sumada a la de los que ya están cargando excederá el límite de carga, por lo cual debe esperar. Cuando el dispositivo está cargado, se procede a *acabar* la carga

Se pide: completar la especificación del recurso compartido que modela el acceso al servicio de Powerbank.

C-TAD Powerbank

OPERACIONES

ACCIÓN cargar : TipoDispositivo[e]

ACCIÓN acabar: TipoDispositivo[e]

SEMÁNTICA

DOMINIO

TIPO Powerbank = $\mathbb{N}$

TIPO TipoDispositivo = Bajo | Alto

INICIAL self = 0

INVARIANTE $0 \leq self \leq maxpower$

CPRE: $(d = \text{Bajo} \Rightarrow self + 5 < maxpower) \wedge$

$(d = \text{Alto} \Rightarrow self + 12 < maxpower)$

cargar (d)

POST: $(d = \text{Bajo} \Rightarrow self = self^{pre} + 5) \wedge$

$(d = \text{Alto} \Rightarrow self = self^{pre} + 12)$

CPRE: cierto

acabar (d)

POST: $(d = \text{Bajo} \Rightarrow self = self^{pre} - 5) \wedge$

$(d = \text{Alto} \Rightarrow self = self^{pre} - 12)$